COMP0222 Simultaneous Localisation and Mapping

Lecture 04: Using Kalman Filters for Nonlinear SLAM

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

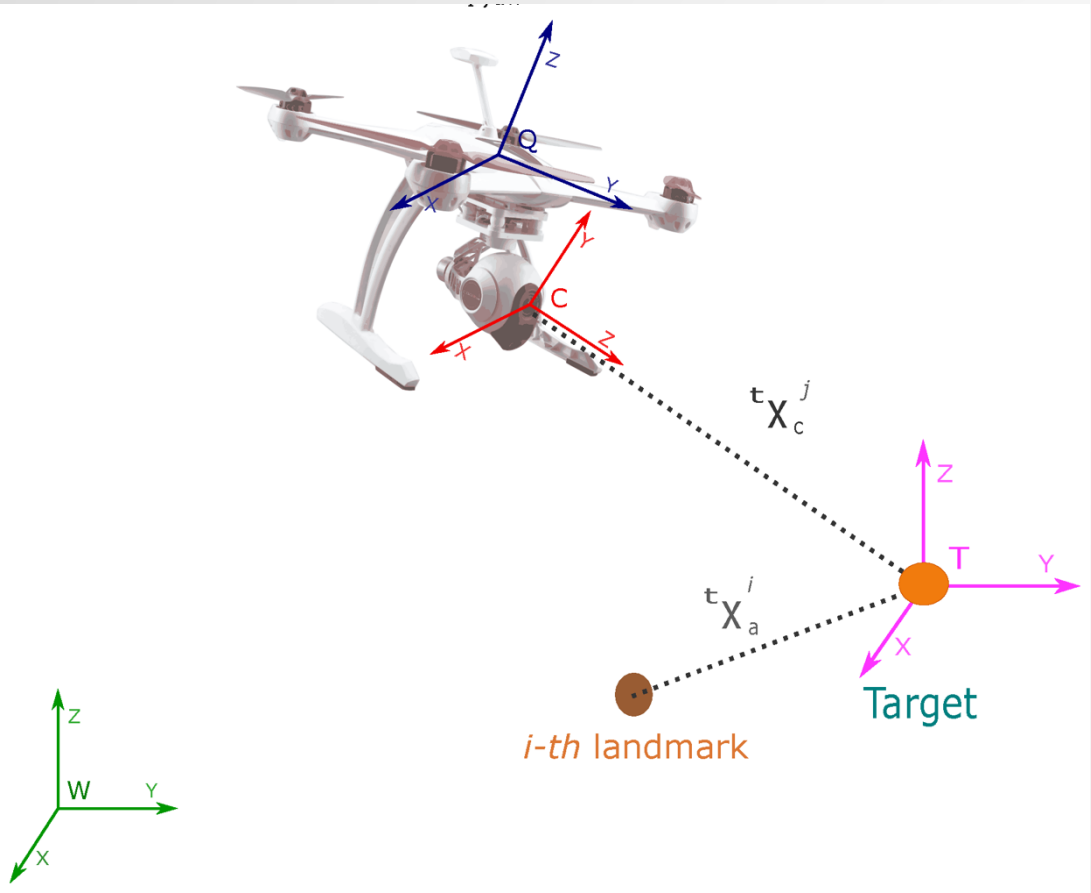
s.saeedi@ucl.ac.uk

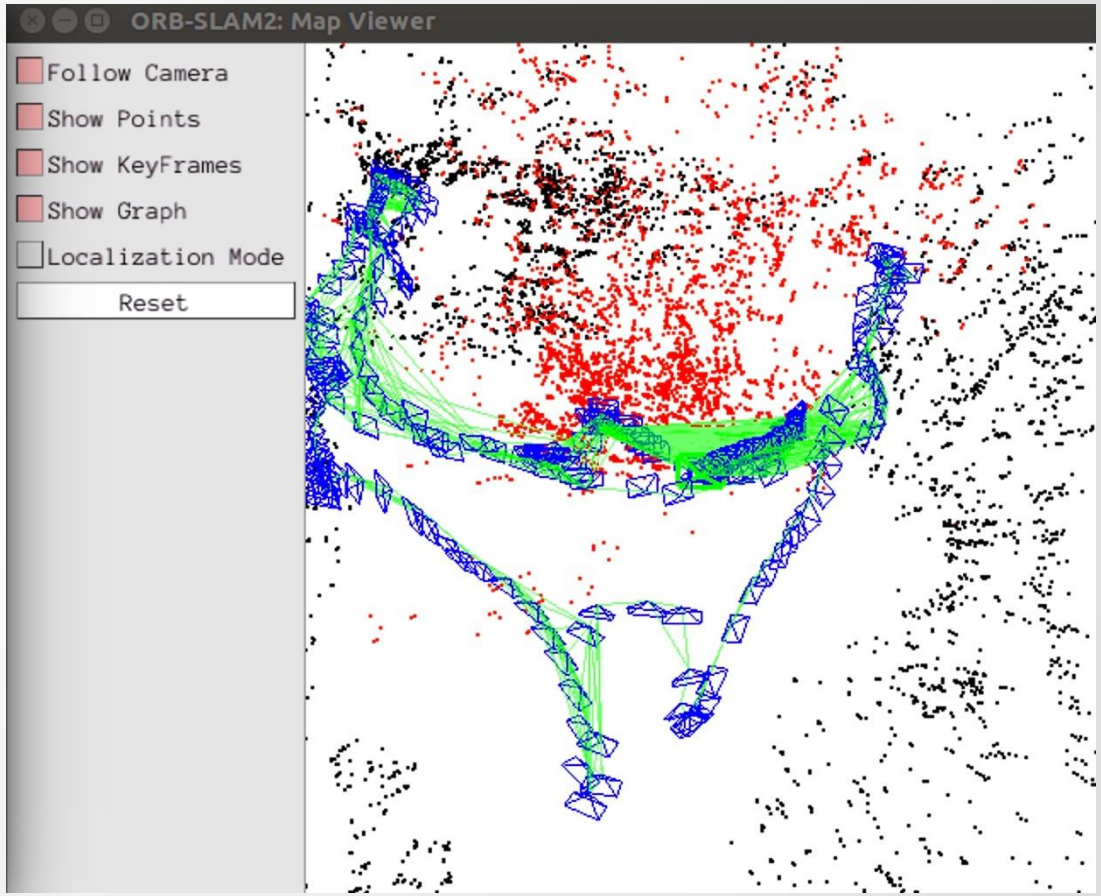
Motivation

- **BeadSLAM and dotbot are linear test cases, and they are not reflective of *any* real SLAM system**

- **Real SLAM systems have *nonlinear models* and are often *very high dimensional***

Motivation


Z
Q
Y
X
Y
C
X
Z
$^{t}X_{c}^{j}$
Z
T
Y
$^{t}X_{a}^{i}$
X
Target
i-th landmark
Z
W
Y
X


ORB-SLAM2: Map Viewer
Follow Camera
Show Points
Show KeyFrames
Show Graph
Localization Mode
Reset

Structure

- EKF-SLAM
- The TriangleBotSLAM Example
- Accuracy of EKF-Based SLAM

EKF-SLAM

- **EKF-SLAM**
- ***The TriangleBotSLAM Example***
- ***Scalability of EKF-Based SLAM***

EKF-SLAM

- The implementation of EKF SLAM is almost the same as linear SLAM
- We use first order Taylor Series expansions of the nonlinear functions to compute the mean and covariance
- We provide a more formal definition of an inverse observation model

Nonlinear Process Model

- **The nonlinear process model is assumed to take the form**

$$\mathbf{x}_{k+1} = \mathbf{f}\left[\mathbf{x}_k, \mathbf{u}_{k+1}, \mathbf{v}_{k+1}\right]$$

Nonlinear Observation Model

- **The observation model is**

$$\mathbf{z}_k^j = \mathbf{h}\left[\mathbf{x}_k, \mathbf{m}^{i_j}, \mathbf{w}_k^j\right]$$

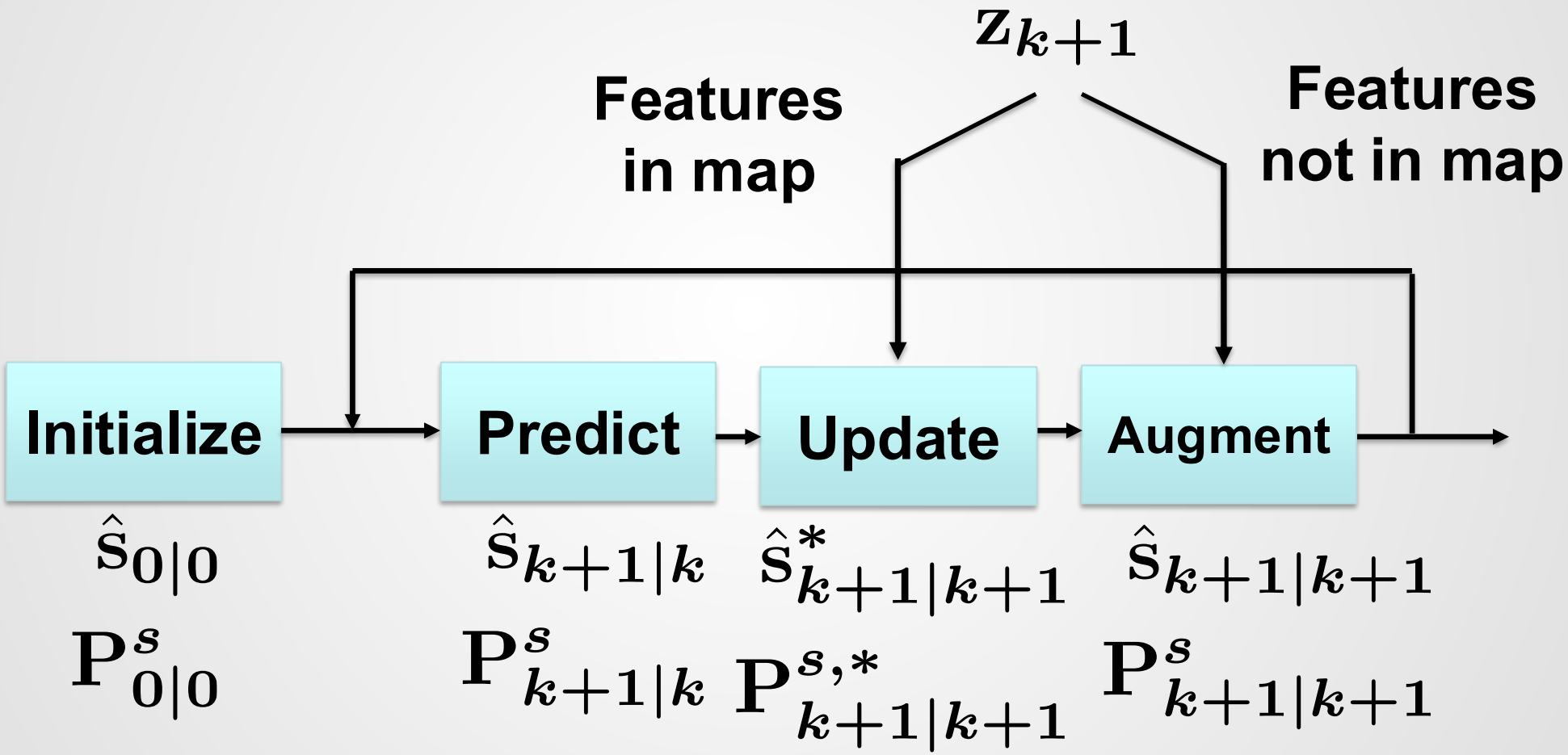
Nonlinear Inverse Observation Model

- **Given an observation and the platform state, the position of the landmark can be estimated,**

$$\mathbf{m}^{i_j} = \mathbf{g}\left[\mathbf{x}_k, \mathbf{z}_k^j, \mathbf{w}_k^j\right]$$

- **When we come to the augmentation step, we'll need the *inverse observation model***

EKF-SLAM


$\mathbf{z}_{k+1}$
Features in map
Features not in map
Initialize
Predict
Update
Augment
$\hat{\mathbf{s}}_{0|0}$
$\mathbf{P}^{s}_{0|0}$
$\hat{\mathbf{s}}_{k+1|k}$
$\mathbf{P}^{s}_{k+1|k}$
$\hat{\mathbf{s}}^{*}_{k+1|k+1}$
$\mathbf{P}^{s,*}_{k+1|k+1}$
$\hat{\mathbf{s}}_{k+1|k+1}$
$\mathbf{P}^{s}_{k+1|k+1}$

Ground Truth State Space Model

- **This contains the robot state, and the state of all the landmarks,**

$$\mathbf{s}_k = \begin{pmatrix} \mathbf{x}_k \\ \mathbf{m} \end{pmatrix} = \begin{pmatrix} \mathbf{x}_k \\ \mathbf{m}^1 \\ \vdots \\ \mathbf{m}^N \end{pmatrix}$$

Kalman Filter State

- Once again, we want to construct and propagate estimates of the form,

$$\hat{\mathbf{s}}_{k|k} = \begin{pmatrix} \hat{\mathbf{x}}_{k|k} \\ \hat{\mathbf{m}}_{k|k} \end{pmatrix} = \begin{pmatrix} \hat{\mathbf{x}}_{k|k} \\ \hat{\mathbf{m}}^{1}_{k|k} \\ \vdots \\ \hat{\mathbf{m}}^{N_k}_{k|k} \end{pmatrix}$$

The Structure of the Covariance Matrix

$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{x}\mathbf{x}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

EKF-SLAM Predicted Mean

$$\hat{\mathbf{s}}_{k+1|k} = \begin{pmatrix} \mathbf{f}\left[\hat{\mathbf{x}}_{k|k}, \mathbf{u}_{k+1}, \mathbf{0}\right] \\ \hat{\mathbf{m}}^{1}_{k|k} \\ \vdots \\ \hat{\mathbf{m}}^{N_k}_{k|k} \end{pmatrix}$$

EKF-SLAM Predicted Covariance

$$\mathbf{P}^{s}_{k+1|k} = \nabla_{\mathbf{x}}\mathbf{f}^{\mathbf{s}}\mathbf{P}^{s}_{k|k}\nabla_{\mathbf{x}}^{\top}\mathbf{f}^{\mathbf{s}} + \nabla_{\mathbf{v}}\mathbf{f}^{\mathbf{s}}\mathbf{Q}_{k+1}\nabla_{\mathbf{v}}\mathbf{f}^{\mathbf{s}}$$

$$\nabla_{\mathbf{x}}\mathbf{f}^{\mathbf{s}} = \begin{bmatrix} \nabla_{\mathbf{x}}\mathbf{f} & 0 & 0 & \cdots & 0 \\ 0 & \mathbf{I} & 0 & \cdots & 0 \\ 0 & 0 & \mathbf{I} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \mathbf{I} \end{bmatrix}, \quad \nabla_{\mathbf{v}}\mathbf{f}^{\mathbf{s}} = \begin{bmatrix} \nabla_{\mathbf{v}}\mathbf{f} \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

EKF-SLAM Update

- The predicted observation is

$$\hat{\mathbf{z}}^{j}_{k+1|k} = \mathbf{h}\left[\hat{\mathbf{x}}^{*}_{k+1|k}, \hat{\mathbf{m}}^{i_j,*}_{k+1|k}, 0\right]$$

- The innovation is

$$\boldsymbol{\nu}^{j}_{k+1|k} = \mathbf{z}^{j}_{k+1} - \hat{\mathbf{z}}^{j}_{k+1|k}$$

EKF-SLAM Update

- **Define the system observation Jacobian,**

$$\nabla_{\mathbf{x}} \mathbf{h}^{\mathbf{s}} = \begin{bmatrix} \nabla_{\mathbf{x}} \mathbf{h} & 0 & \cdots & 0 & \nabla_{\mathbf{m}} \mathbf{h} & \cdots & 0 \end{bmatrix}$$

EKF-SLAM Update

- The update given that we've observed a landmark e is given by

$$\hat{\mathbf{s}}^{*}_{k+1|k+1} = \hat{\mathbf{s}}^{*}_{k+1|k} + \mathbf{W}^{e}_{k+1|k}\boldsymbol{\nu}^{e}_{k+1|k}$$

$$\mathbf{P}^{s,*}_{k+1|k+1} = \mathbf{P}^{s,*}_{k+1|k} - \mathbf{W}^{e}_{k+1|k}\mathbf{S}^{e}_{k+1|k}\mathbf{W}^{e,\top}_{k+1|k}$$

EKF-SLAM Update

- **The matrices for the Kalman filter are**

$$\mathbf{W}^{e}_{k+1|k} = \mathbf{C}^{e}_{k+1|k}\left(\mathbf{S}^{e}_{k+1|k}\right)^{-1}$$

$$\mathbf{C}^{e}_{k+1|k} = \mathbf{P}^{s}_{k+1|k}\boldsymbol{\nabla}^{\top}_{\mathbf{x}}\mathbf{h}^{\mathbf{s}}$$

$$\mathbf{S}^{e}_{k+1|k} = \boldsymbol{\nabla}_{\mathbf{x}}\mathbf{h}^{\mathbf{s}}\,\mathbf{P}^{s}_{k+1|k}\boldsymbol{\nabla}^{\top}_{\mathbf{x}}\mathbf{h}^{\mathbf{s}} + \boldsymbol{\nabla}_{\mathbf{w}}\mathbf{h}\,\mathbf{R}_{k+1}\boldsymbol{\nabla}^{\top}_{\mathbf{w}}\mathbf{h}$$

EKF-SLAM Augmentation

- **Suppose we augment with a new landmark n**

$$\hat{\mathbf{s}}_{k+1|k+1} = \mathbf{g}^{\mathbf{s}}\left[\hat{\mathbf{x}}^{*}_{k+1|k+1}, \mathbf{z}^{n}_{k+1}\right] = \begin{pmatrix} \hat{\mathbf{x}}^{*}_{k+1|k+1} \\ \hat{\mathbf{m}}^{1,*}_{k+1|k+1} \\ \vdots \\ \hat{\mathbf{m}}^{N_k,*}_{k+1|k+1} \\ \mathbf{g}\left[\hat{\mathbf{x}}^{*}_{k+1|k+1}, \mathbf{z}^{n}_{k+1}, \mathbf{0}\right] \end{pmatrix}$$

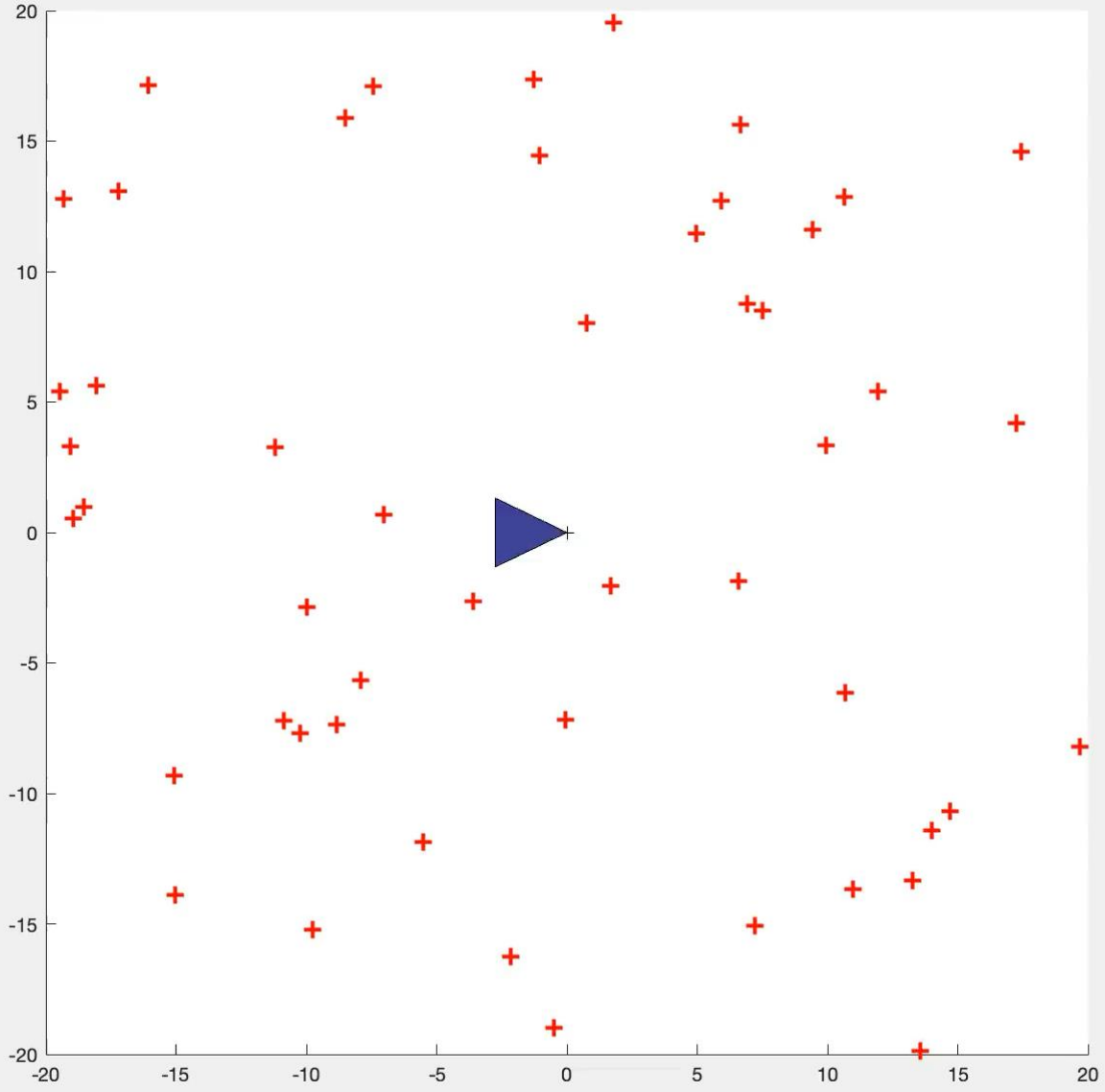
EKF-SLAM Augmentation Covariance

$$\mathbf{P}^{s}_{k+1|k+1} = \nabla_{\mathbf{x}}\mathbf{g}^{\mathbf{s}}\mathbf{P}^{s,*}_{k+1|k+1}\nabla_{\mathbf{x}}^{\top}\mathbf{g}^{\mathbf{s}} + \nabla_{\mathbf{w}}\mathbf{g}^{\mathbf{s}}\mathbf{R}^{n}_{k+1}\nabla_{\mathbf{w}}^{\top}\mathbf{g}^{\mathbf{s}}$$

$$\nabla_{\mathbf{x}}\mathbf{g}^{\mathbf{s}} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{I} & \mathbf{0} \\ \nabla_{\mathbf{x}}\mathbf{g} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \nabla_{\mathbf{w}}\mathbf{g}^{\mathbf{s}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \\ \nabla_{\mathbf{w}}\mathbf{g} \end{bmatrix}$$

The TriangleBotSLAM Example

- *EKF-SLAM*
- **The TriangleBotSLAM Example**
- *Accuracy of EKF-Based SLAM*


20
15
10
5
0
-5
-10
-15
-20
-20
-15
-10
-5
0
5
10
15
20

TriangleBot Platform and Map Models

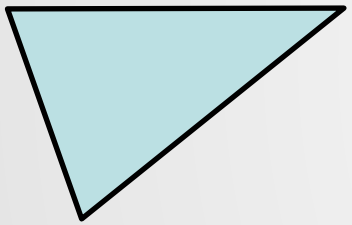
- **The platform state is**

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \theta_k \end{bmatrix}^\top$$

- **The state of the ith landmark is**

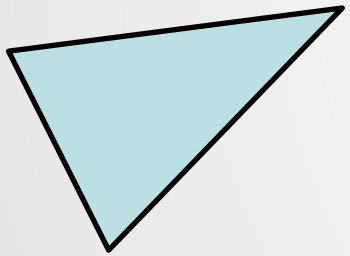
$$\mathbf{m}^i = \begin{bmatrix} x^i \\ y^i \end{bmatrix}$$

Process Model



$$x_{k+1} = x_k + s_{k+1} \Delta T \cos \theta_k$$
$$y_{k+1} = y_k + s_{k+1} \Delta T \sin \theta_k$$
$$\theta_{k+1} = \theta_k + \Delta T \omega_{k+1}$$

Process Noise



$$\mathbf{v}_k = \begin{bmatrix} v_k^l \\ v_k^t \\ v_k^\omega \end{bmatrix}$$

Process Model

- **Putting these together, we can simplify this lot to**

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta T \mathrm{M}\left[\theta_k\right]\left(\mathbf{u}_{k+1} + \mathbf{v}_{k+1}\right)$$

$$\mathrm{M}\left[\theta_k\right] = \begin{bmatrix} \cos\theta_k & -\sin\theta_k & 0 \\ \sin\theta_k & \cos\theta_k & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{u}_k = \begin{bmatrix} s_k \\ 0 \\ \omega_k \end{bmatrix} \quad \mathbf{v}_k = \begin{bmatrix} v_k^l \\ v_k^t \\ v_k^\omega \end{bmatrix}$$

Process Model Jacobian

- **The Jacobians of the process model are**

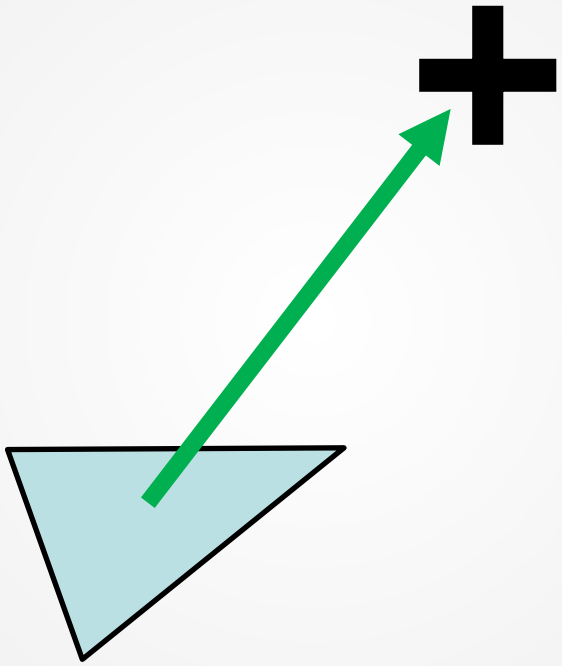
$$\nabla_{\mathbf{x}}\mathbf{f} = \begin{bmatrix} 1 & 0 & -s_{k+1}\Delta T \sin\theta_k \\ 0 & 1 & s_{k+1}\Delta T \cos\theta_k \\ 0 & 0 & 1 \end{bmatrix}, \nabla_{\mathbf{v}}\mathbf{f} = \Delta T \mathrm{M}\left[\theta_k\right]$$

Observation Model

- The robot can measure the range and bearing to all targets within a detection range r_{max} of the robot
- The measurement is

$$\mathbf{z}_k^j = \begin{bmatrix} r_k^j \\ \beta_k^j \end{bmatrix}$$

Observation Model



Observation Model

- Recall the coordinates of the ith landmark are

$$\mathbf{m}^i = \begin{bmatrix} x^i \\ y^i \end{bmatrix}$$

- The displacement, relative to the platform, is

$$\Delta x_k^j = x^{i_j} - x_k$$

$$\Delta y_k^j = y^{i_j} - y_k$$

Observation Model [REDRAW]

- **The measurements are given by**

$$r_k^j = \sqrt{\left(\Delta x_k^j\right)^2 + \left(\Delta y_k^j\right)^2} + \mathbf{w}_{k+1}^{r^j}$$

$$\beta_k^j = \arctan\left(\frac{\Delta y_k^j}{\Delta x_k^j}\right) - \theta_k + \mathbf{w}_{k+1}^{\beta^j}$$

Observation Model Jacobian

- **The observation model Jacobian is a function of both the platform estimate and the landmark estimate**

$$\nabla_{\mathbf{s}}\mathbf{h} = \begin{bmatrix} \nabla_{\mathbf{x}}\mathbf{h} & \mathbf{0} & \cdots & \mathbf{0} & \nabla_{\mathbf{m}}\mathbf{h} & \mathbf{0} & \cdots & \mathbf{0} \end{bmatrix}$$

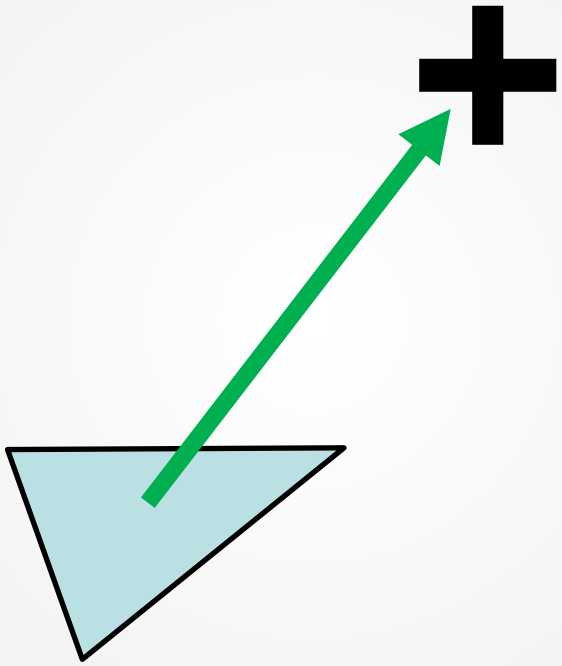
Observation Model Jacobian

- **Taking derivatives, you can show that**

$$\nabla_{\mathbf{x}}\mathbf{h} = \begin{bmatrix} -\Delta x_k^i / r_k^i & -\Delta y_k^i / r_k^i & 0 \\ \Delta y_k^i / (r_k^i)^2 & \Delta x_k^i / (r_k^i)^2 & -1 \end{bmatrix}$$

$$\nabla_{\mathbf{m}}\mathbf{h} = \begin{bmatrix} \Delta x_k^i / r_k^i & \Delta y_k^i / r_k^i \\ -\Delta y_k^i / (r_k^i)^2 & -\Delta x_k^i / (r_k^i)^2 \end{bmatrix}$$

Inverse Observation Model



Inverse Observation Model

- **The inverse observation model is**

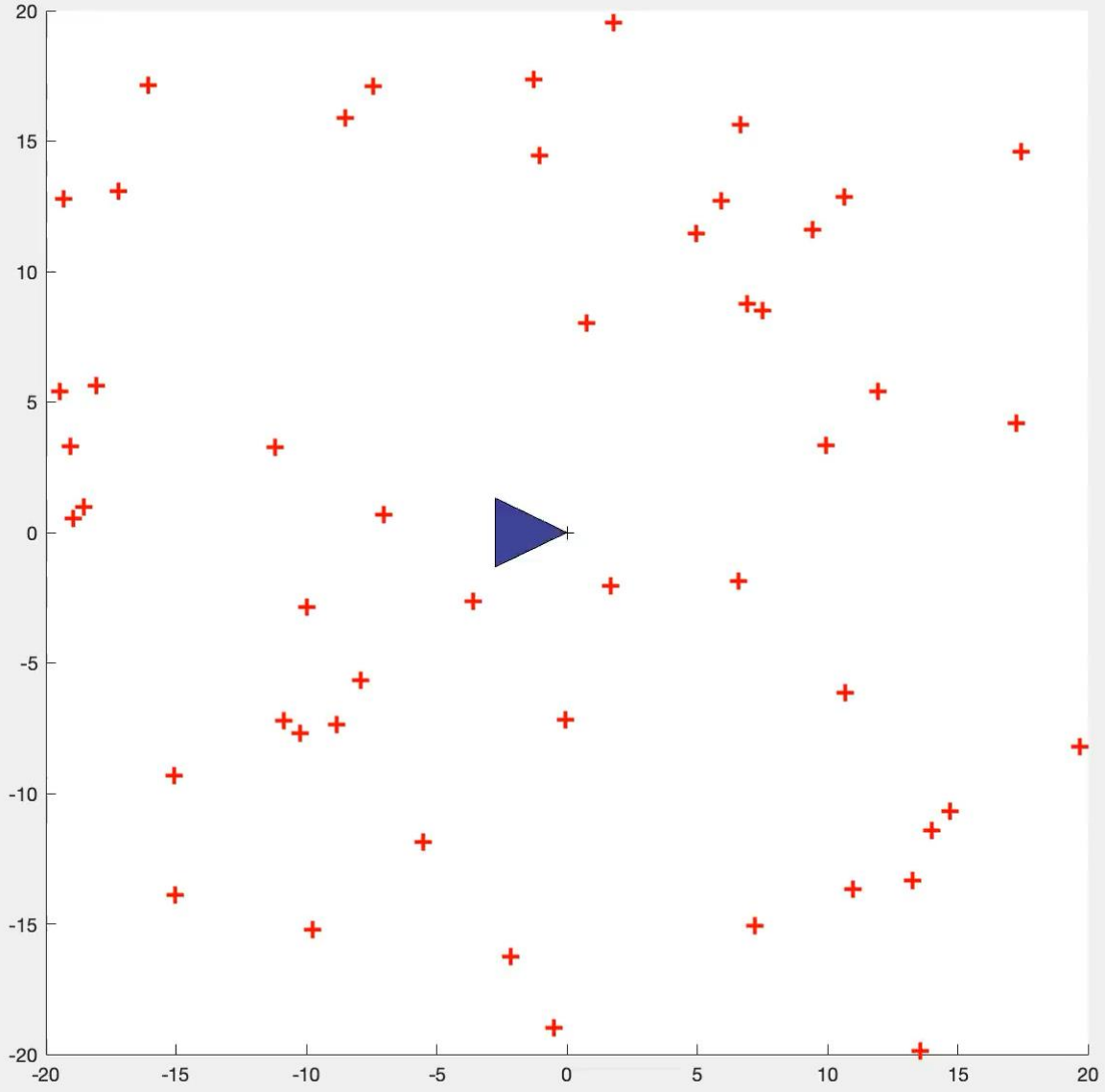
$$\hat{\mathbf{m}}^{i}_{k+1|k+1} = \mathbf{g}\left[\hat{\mathbf{s}}^{*}_{k+1|k}, \mathbf{z}^{i}_{k+1}\right]$$

$$= \begin{bmatrix} \hat{x}^{*}_{k+1|k+1} + r^{i}_{k}\cos\left(\hat{\theta}^{*}_{k+1|k+1} + \beta^{i}_{k}\right) \\ \hat{y}^{*}_{k+1|k+1} + r^{i}_{k}\sin\left(\hat{\theta}^{*}_{k+1|k+1} + \beta^{i}_{k}\right) \end{bmatrix}$$

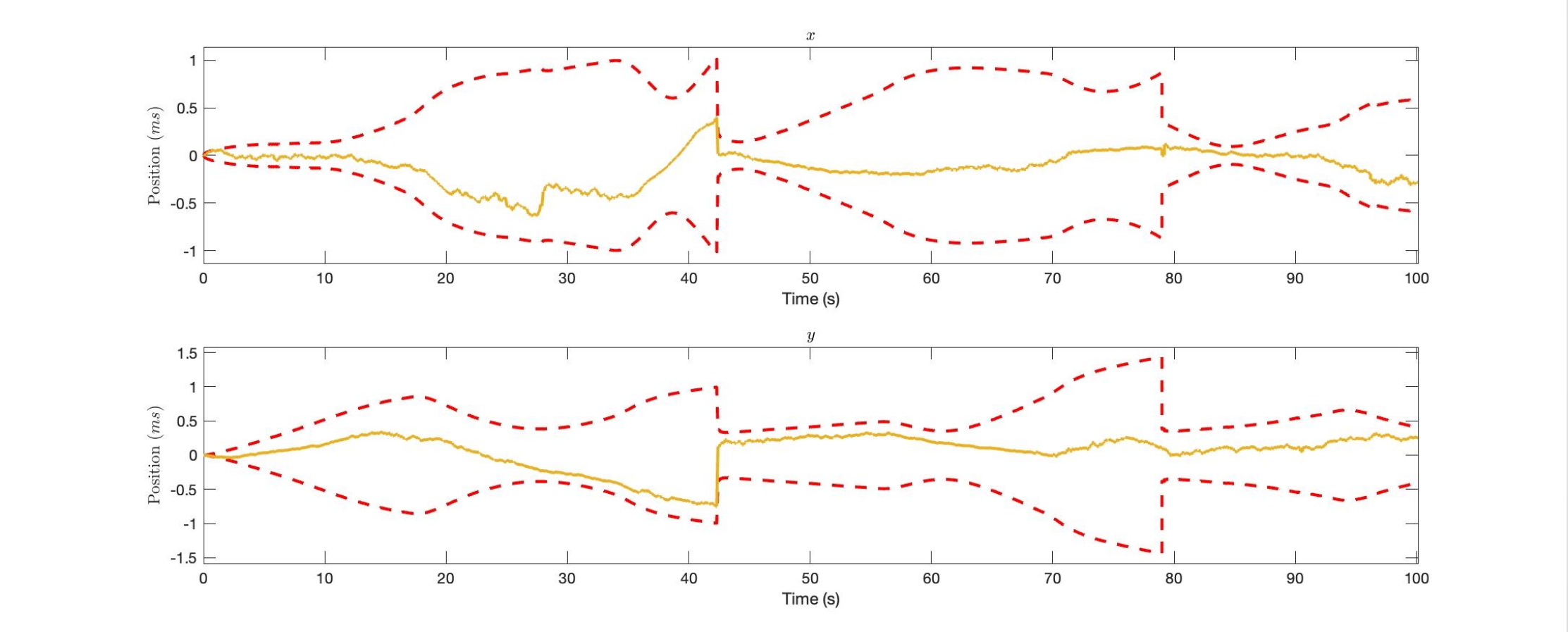
Jacobians of the Inverse Observation Model

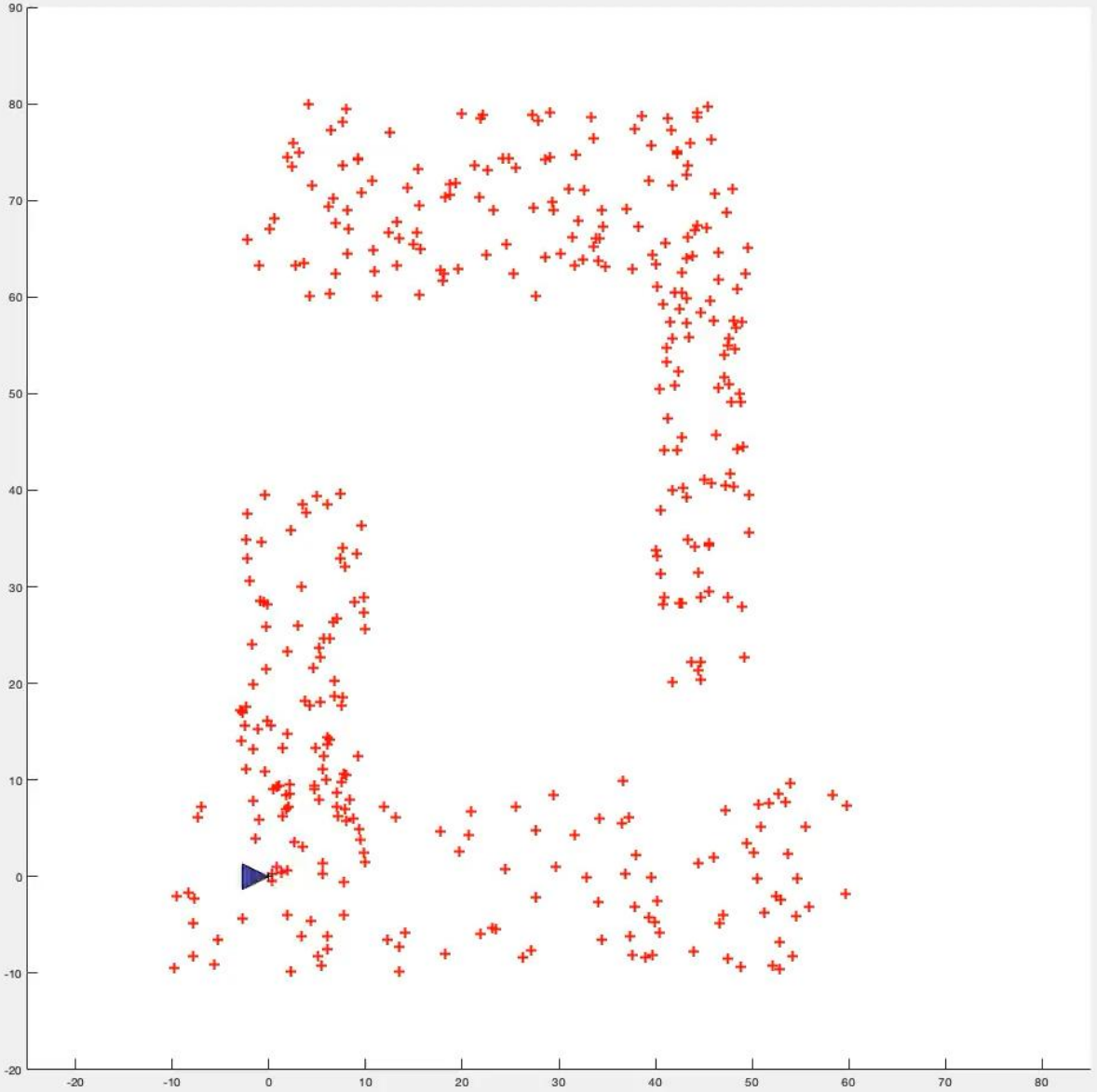
$$\nabla_{\mathbf{x}}\mathbf{g} = \begin{bmatrix} 1 & 0 & -r_k^i \sin\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) \\ 0 & 1 & r_k^i \cos\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) \end{bmatrix}$$

$$\nabla_{\mathbf{w}}\mathbf{g} = \begin{bmatrix} \cos\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) & -r_k^i \sin\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) \\ \sin\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) & r_k^i \cos\left(\hat{\theta}^*_{k+1|k+1} + \beta_k^i\right) \end{bmatrix}$$

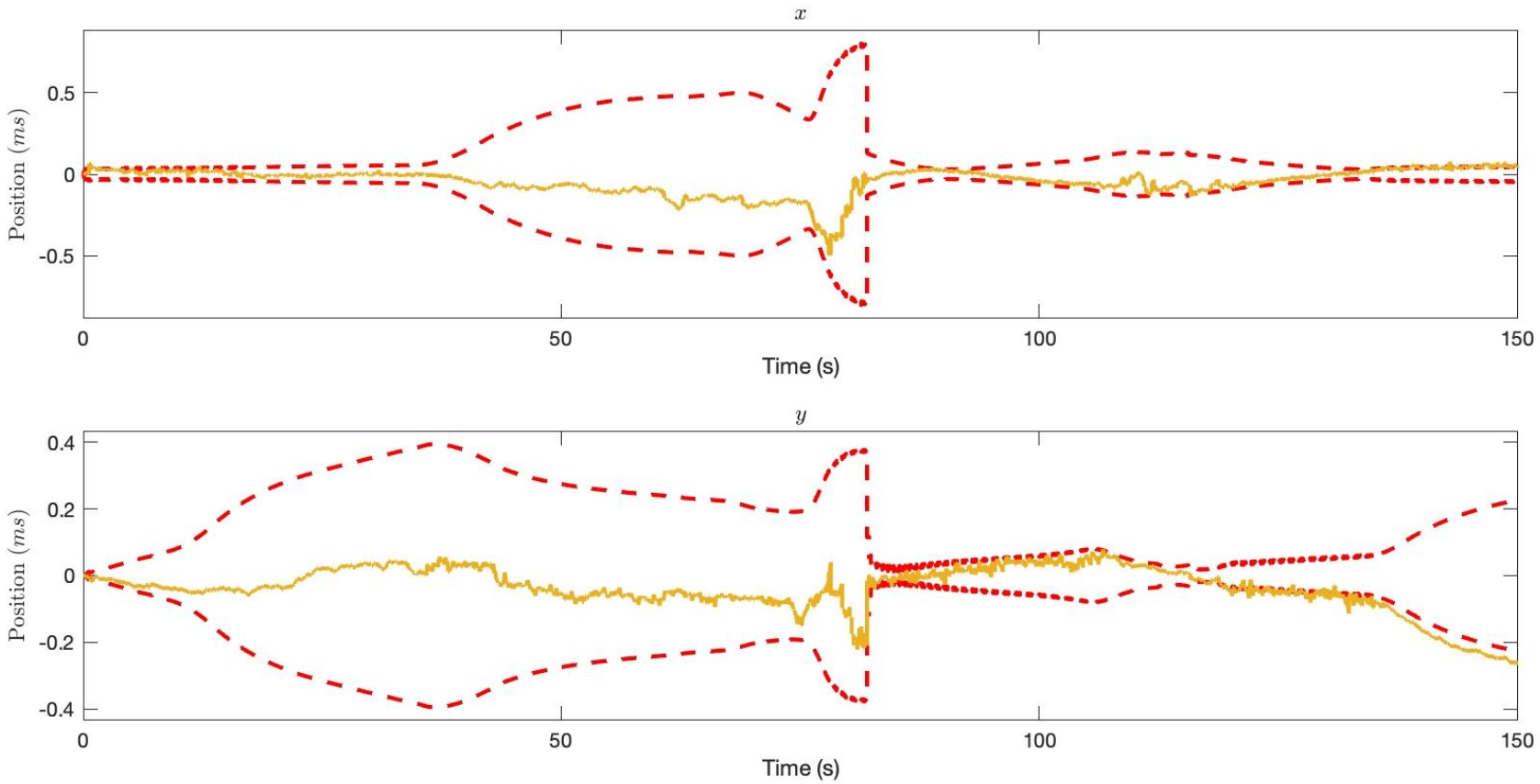

20
15
10
5
0
-5
-10
-15
-20
-20
-15
-10
-5
0
5
10
15
20

Plot of the Error in x, y Over Time




90
80
70
60
50
40
30
20
10
0
-10
-20
-20
-10
0
10
20
30
40
50
60
70
80

Plot of the Error in x, y Over Time

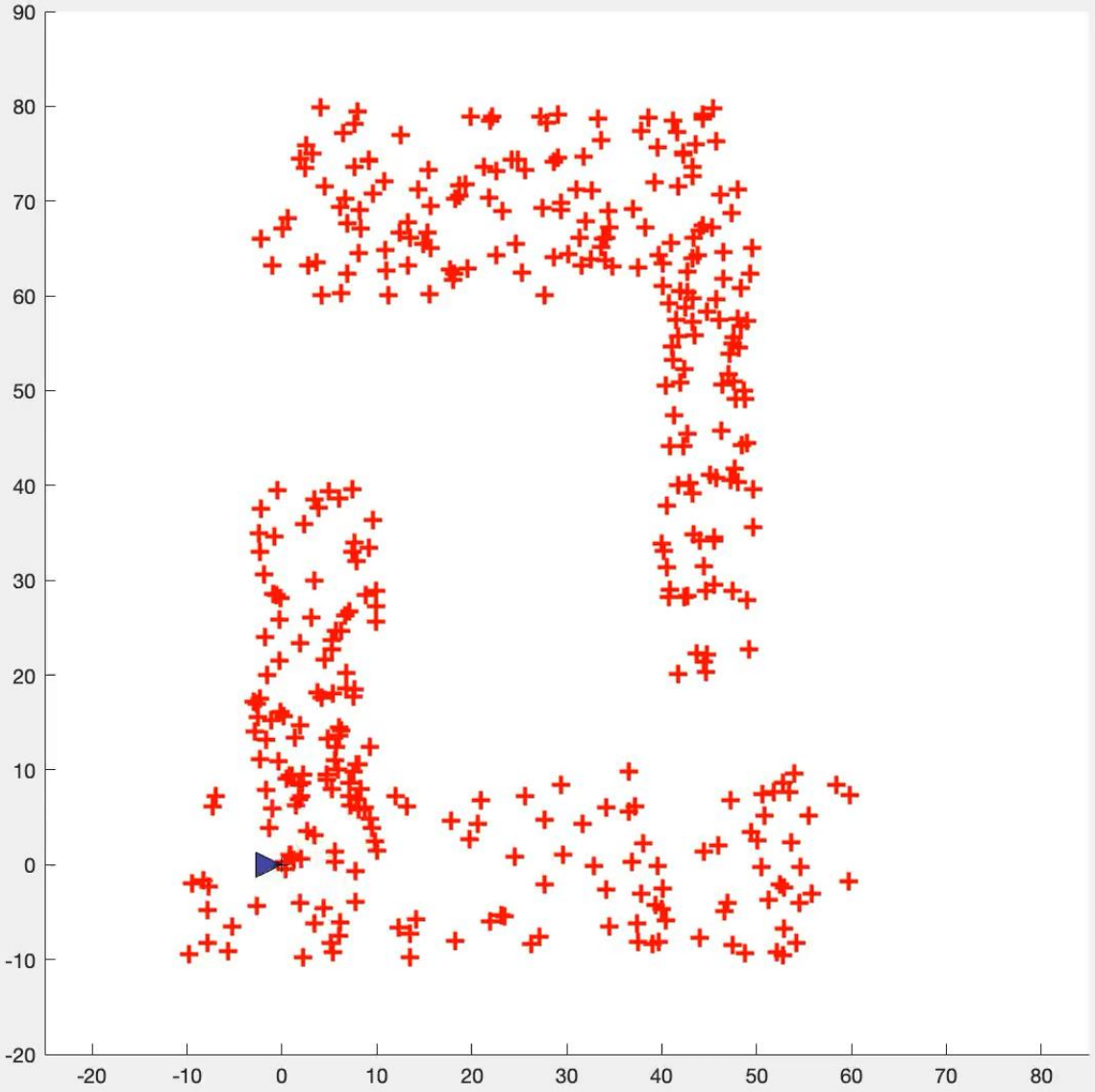


So… It All Works, Right?

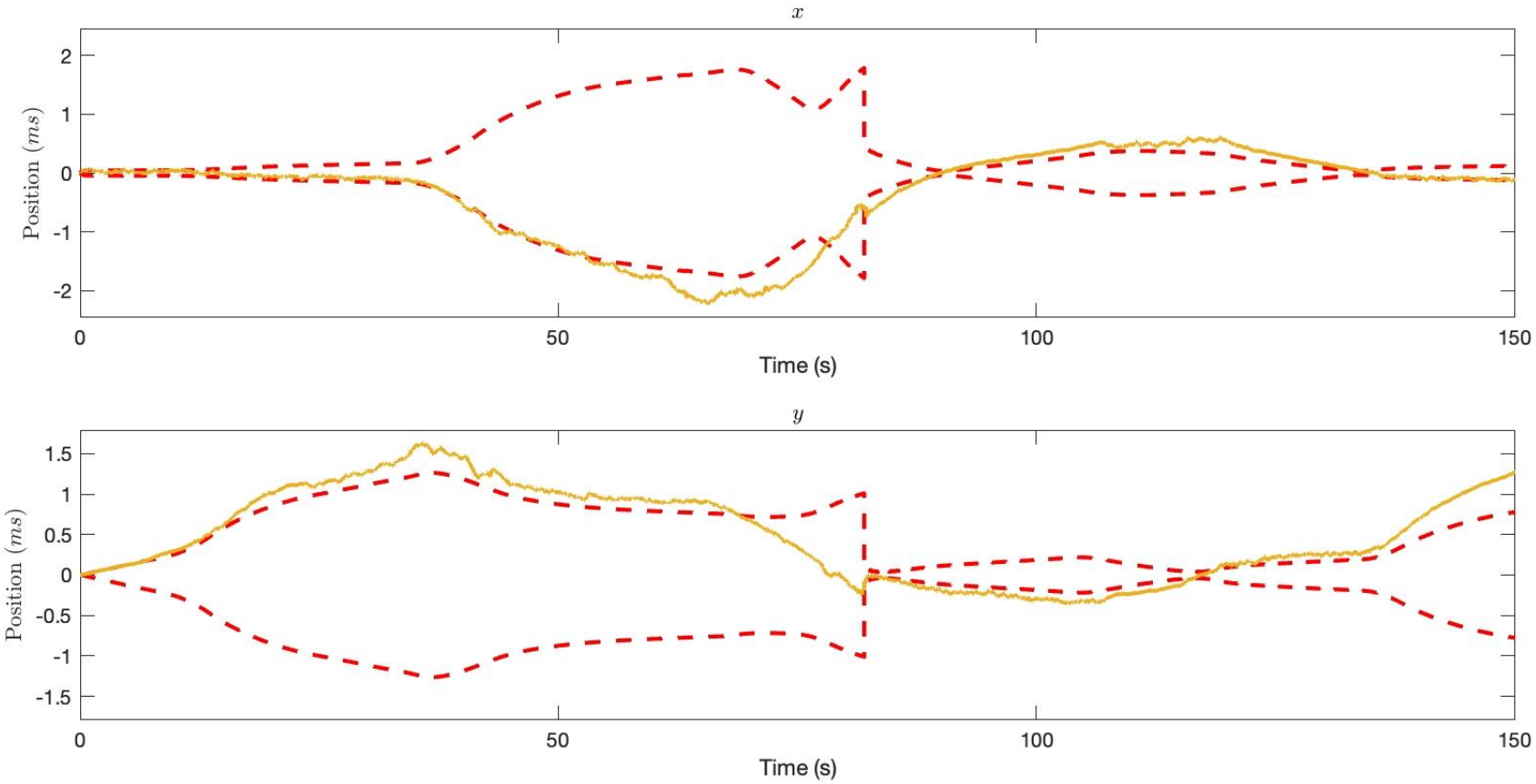
- So far, the results suggest that EKF-SLAM is working really well
- If that's the case why do we need to get into this factor graph stuff at all?

Accuracy of EKF-Based SLAM

- *EKF-SLAM*
- *The TriangleBotSLAM Example*
- Accuracy of EKF-Based SLAM


90
80
70
60
50
40
30
20
10
0
-10
-20
-20
-10
0
10
20
30
40
50
60
70
80

Estimation Error



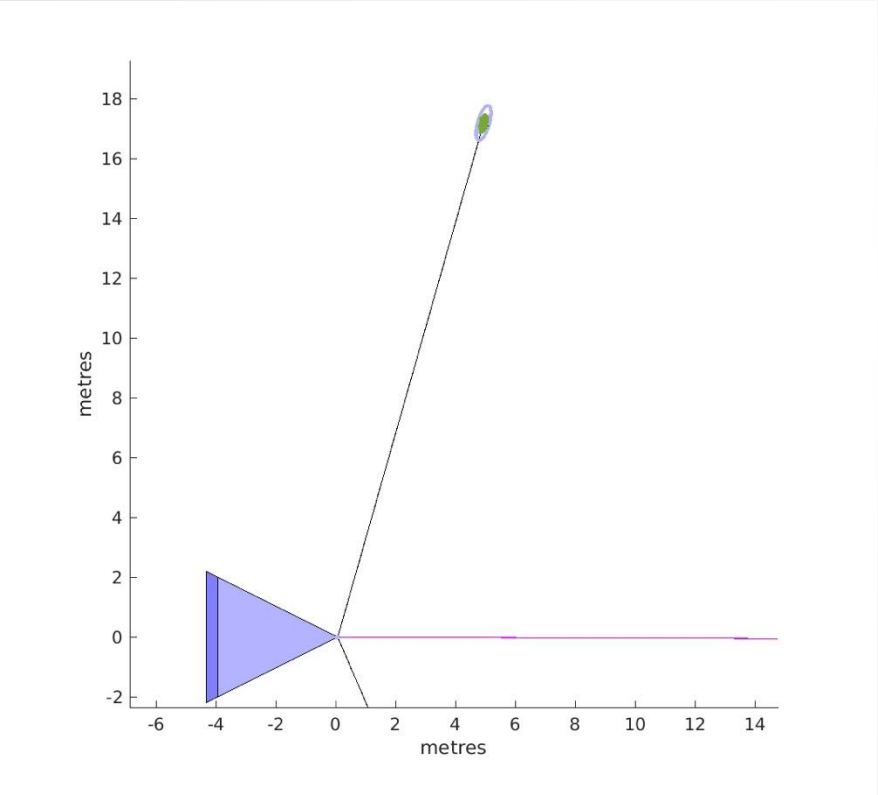
Why Did EKF-SLAM Drift Off?

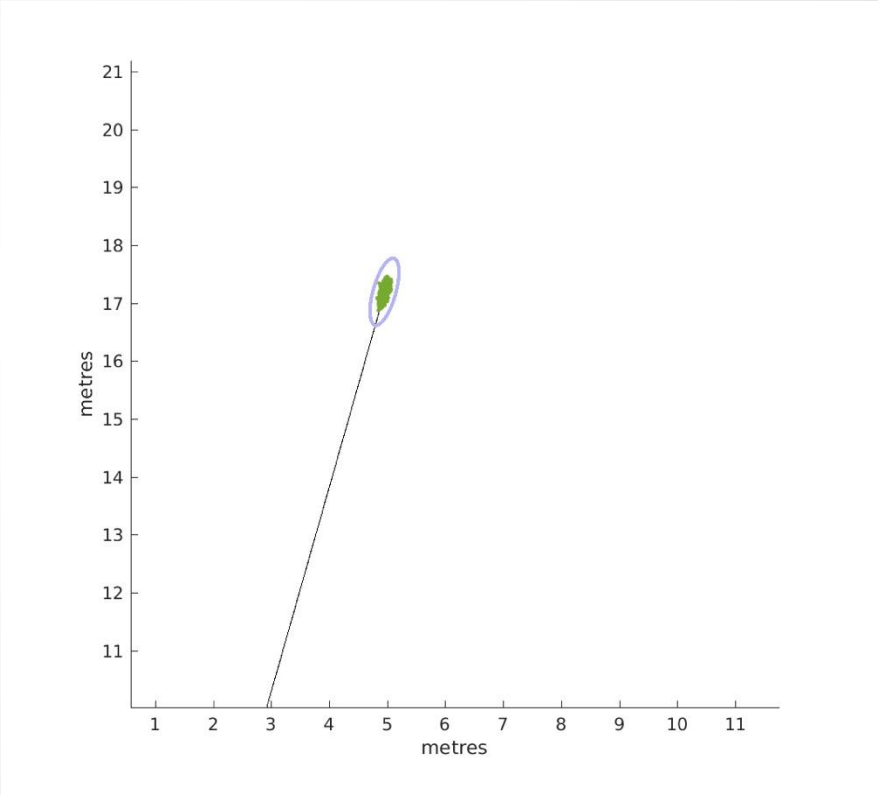
- The Kalman filter critically depends upon the correct relationship between the state estimate and the covariance matrix
- These are clearly going wrong
- Why is this the case?

What do Angular Errors Look Like?

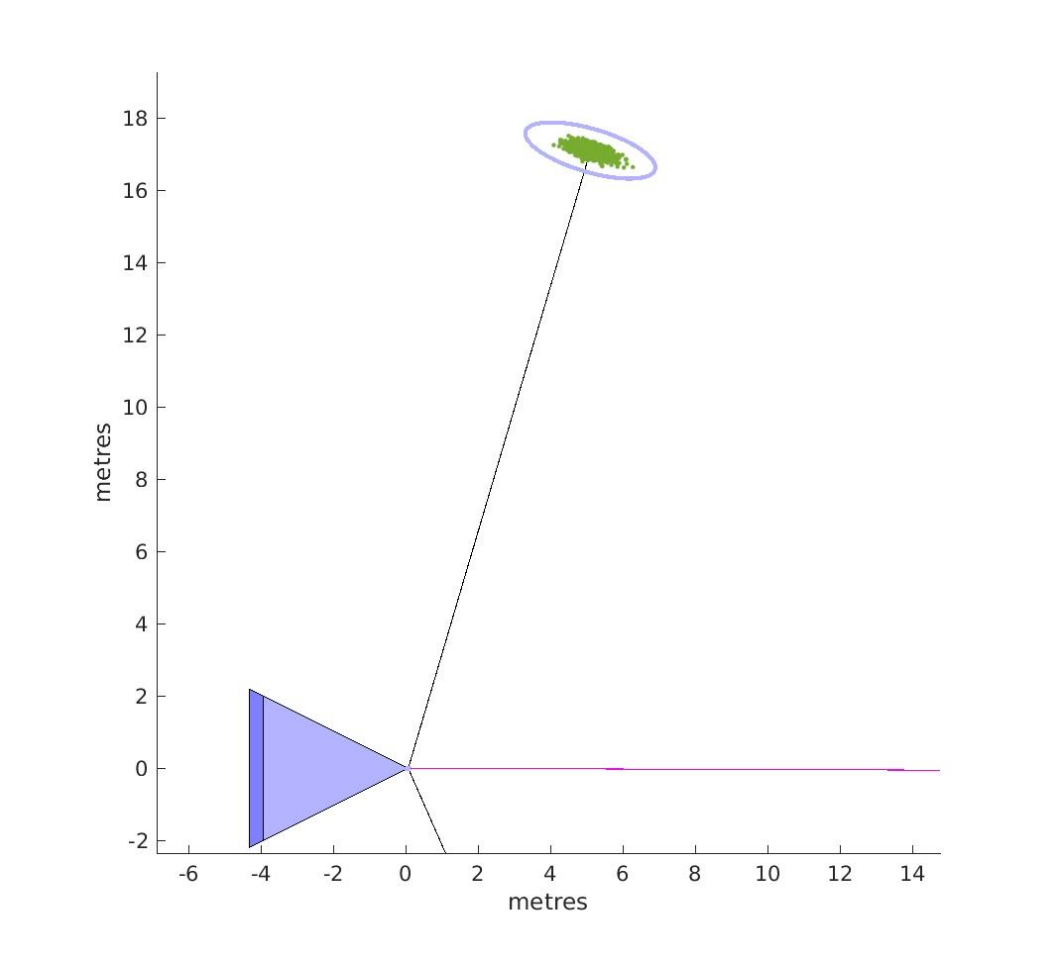
- We can get a sense of what they are doing by sampling particles to illustrate the distributions
- We'll look at particle sampling later, but the main thing is that there's a cloud of points which give a sense of a shape of what the probability distribution looks like

Visualising Covariance (0.5 Deg Std)

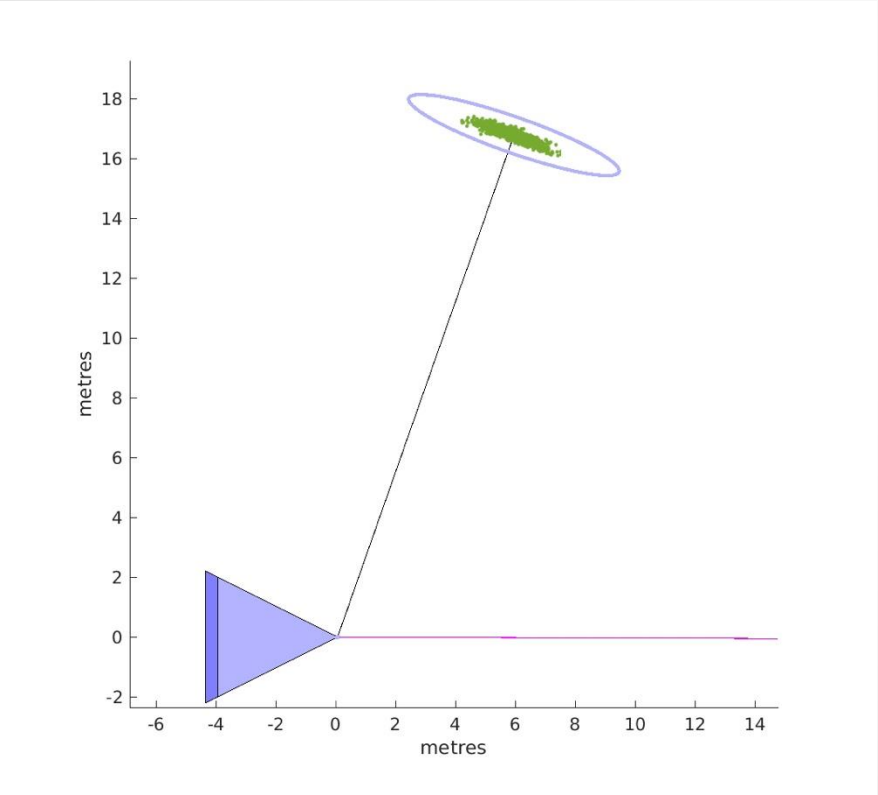


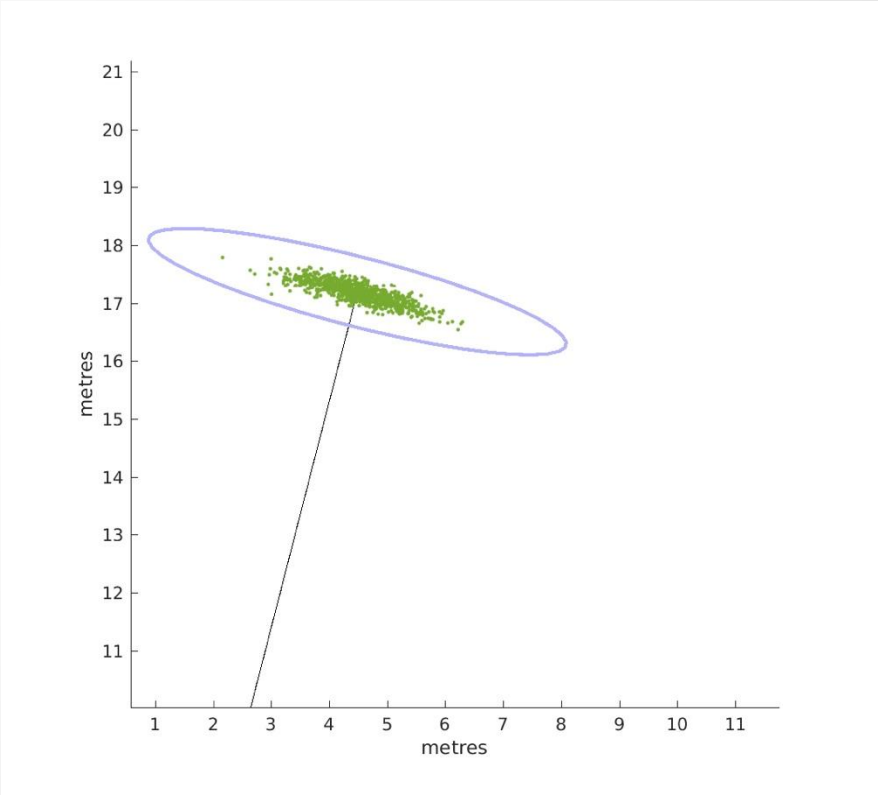


Visualising Covariance (1 Deg Std)

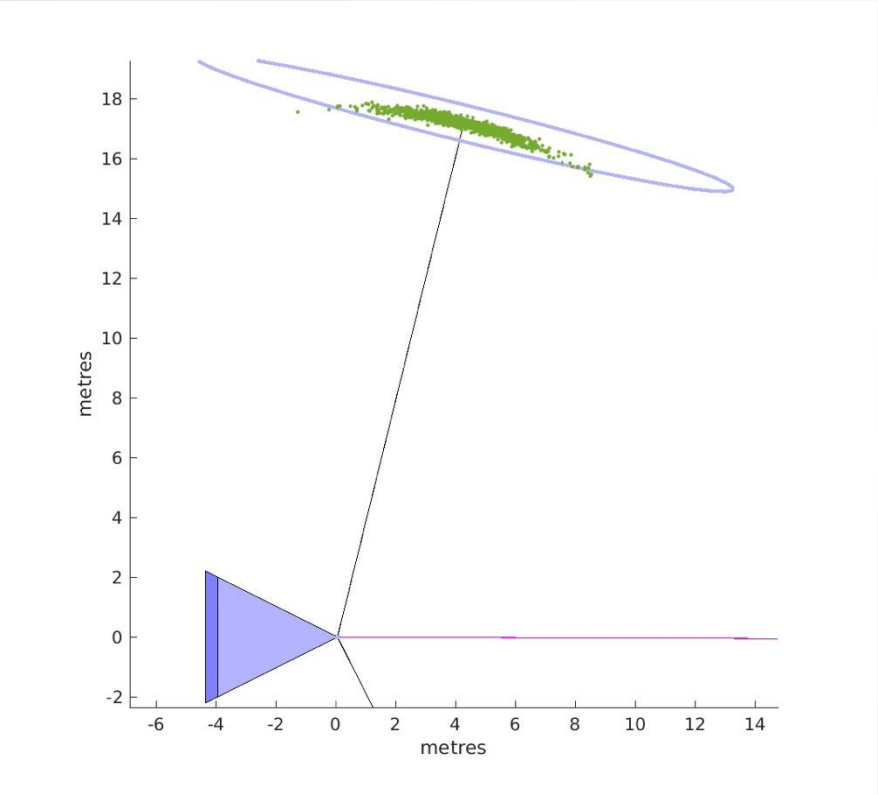


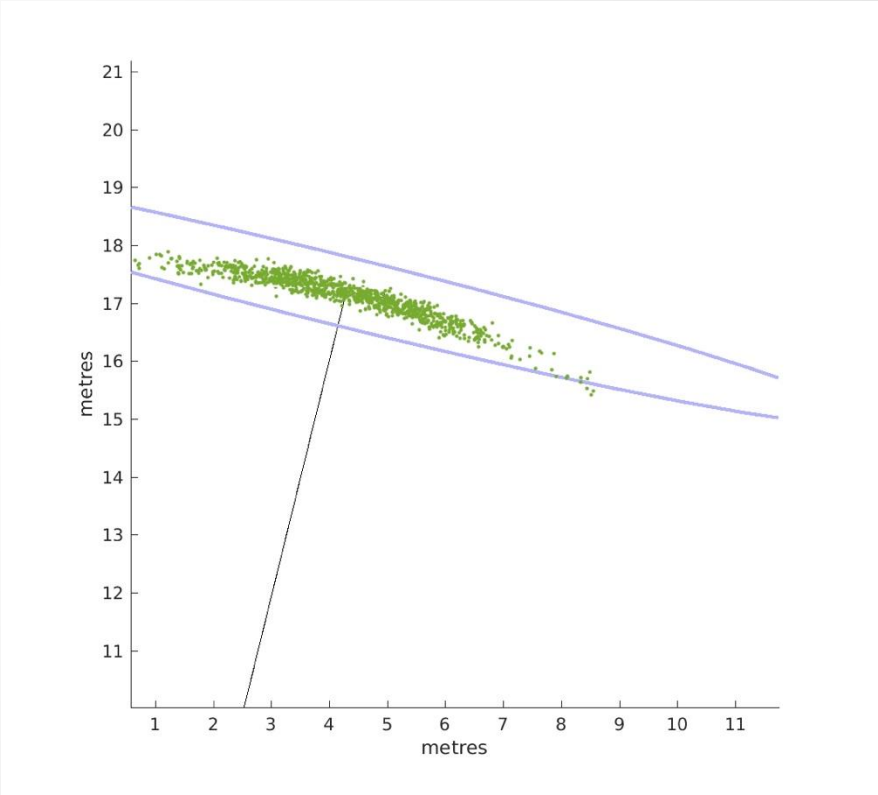
Visualising Covariance (2 Deg Std)





Visualising Covariance (5 Deg Std)

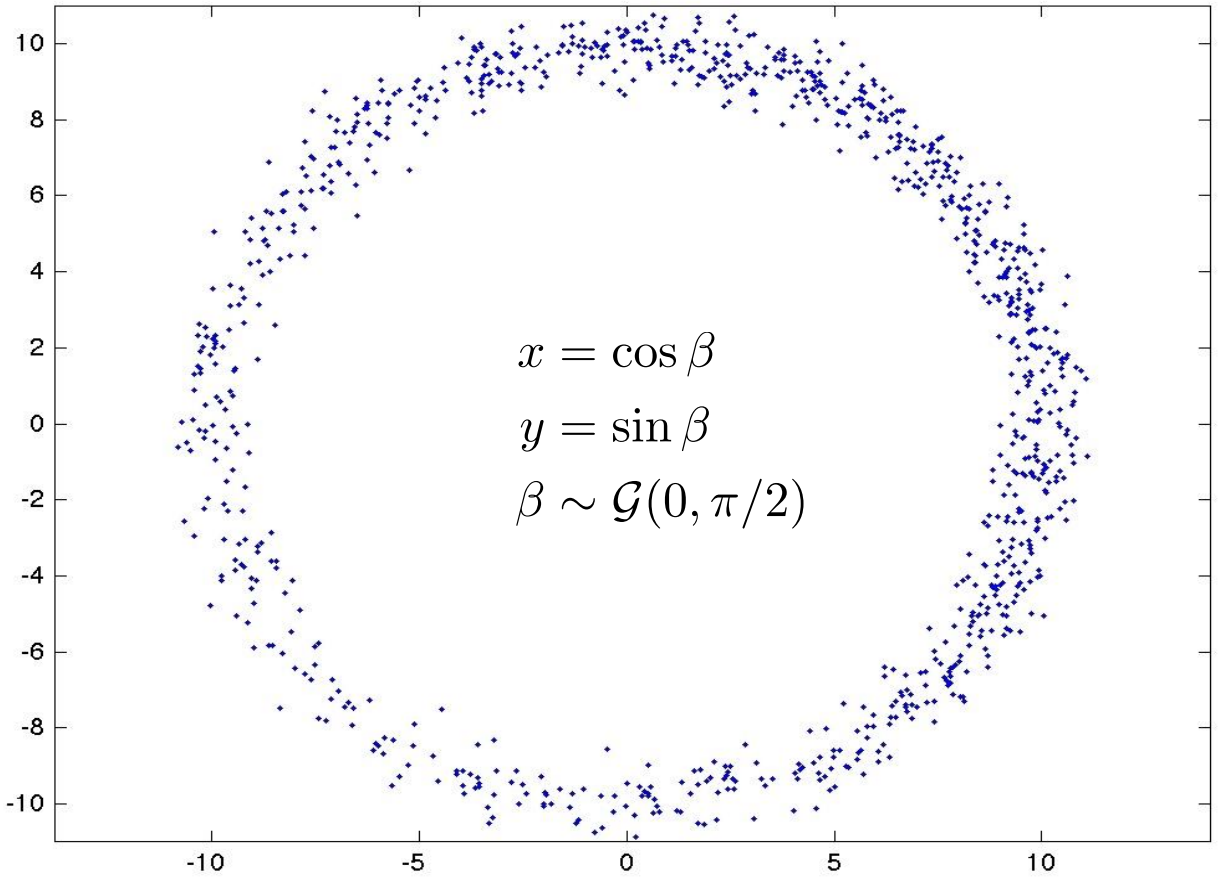




Attempts to Fix the Problem

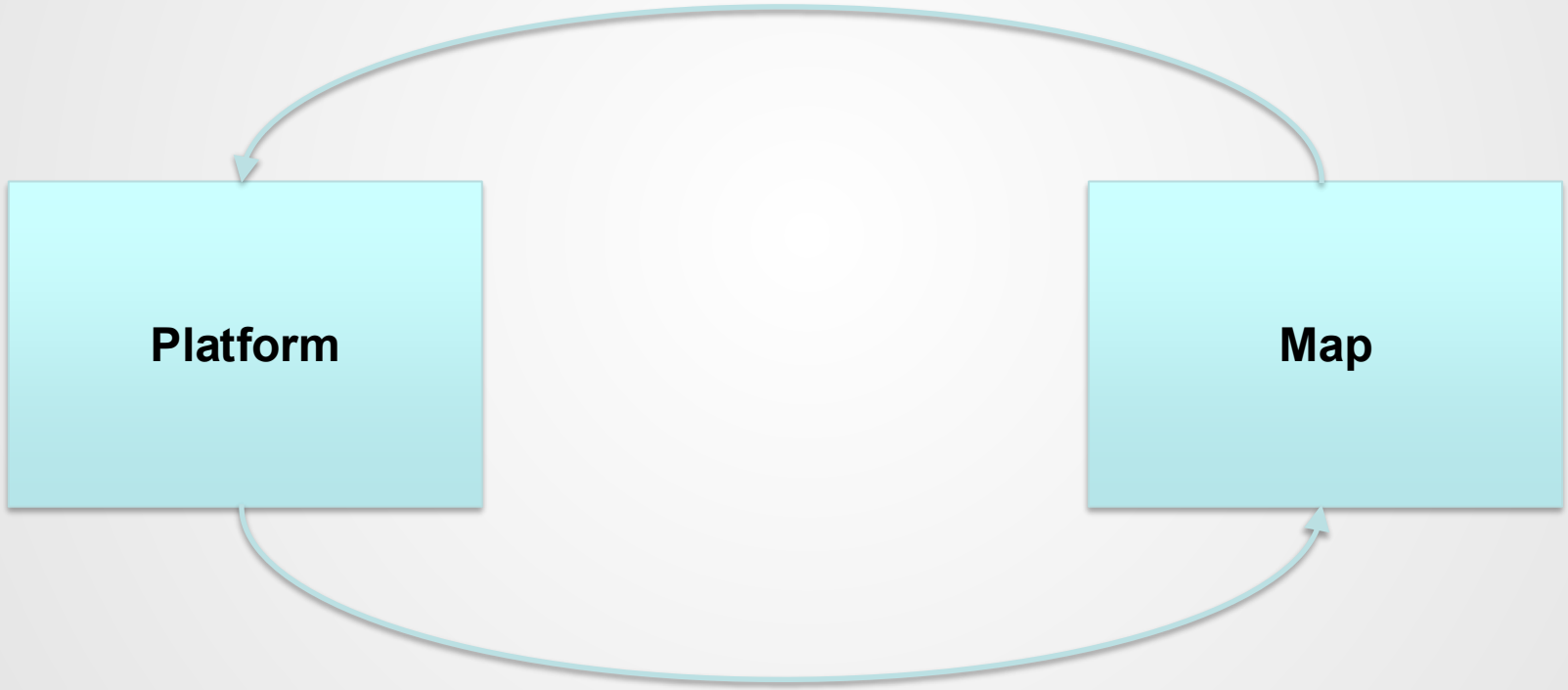
- Play around with the noise matrices to tune the filter
- Use higher order Taylor Series Expansions to better approximate the results
- Exact analytical solutions given Gaussian assumptions
- However, none of these approaches *worked*
- The reason is that *correlations are not good enough* to measure how angular dependencies propagate through the system

Correlations and Angular Dependencies



Self-Mapping Chicken and Egg Problem

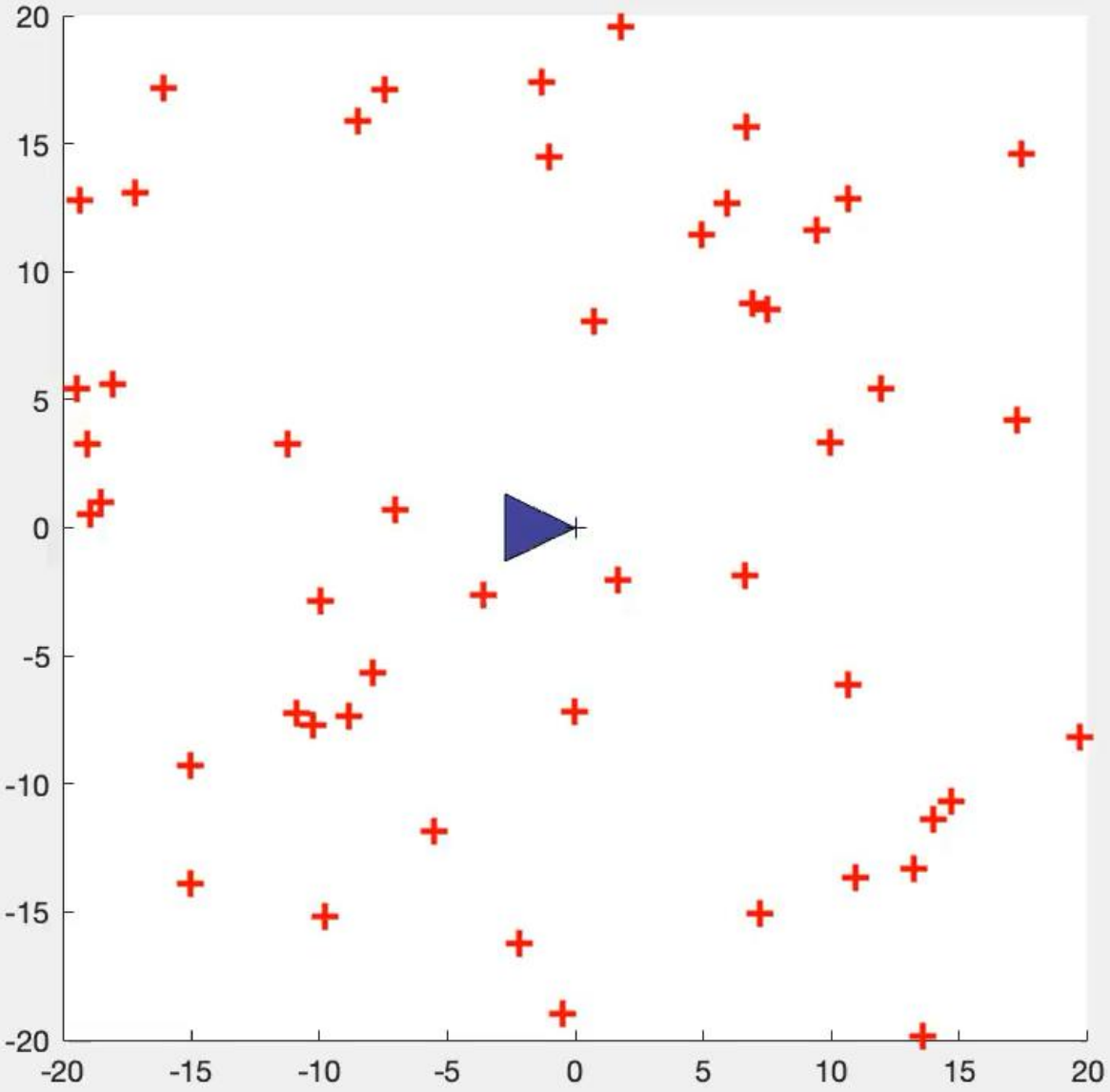
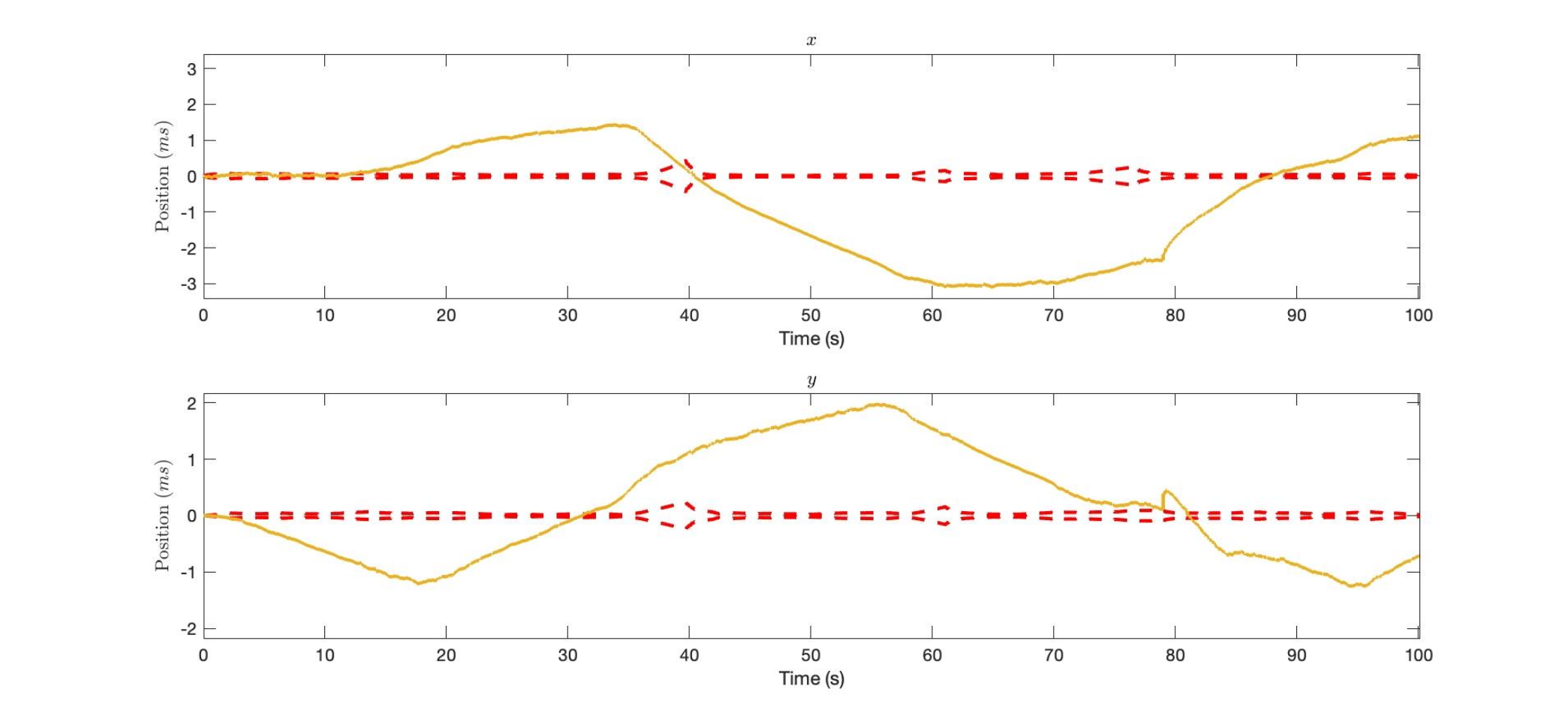
The platform uses the map to localise



The map uses the localised platform to build itself

Doing Better than the Kalman Filter

- One way to do better is to try to develop systems which use the full equations from Bayes rule and probability to solve the problem
- The optional Lecture 05 describes how you would do this
- However, the bottom line is that it is *impossible* to implement the optimal solution
- Therefore, better estimators have to be used
- The most widely used and successful one is a *factor graph*


20
15
10
5
0
-5
-10
-15
-20
-20
-15
-10
-5
0
5
10
15
20

Estimation Error